

Training Logic Walkthrough

Training Logic Walkthrough

Table of Contents

1. Big Picture
2. Background: What Is a Language Model?
3. Hyperparameters
4. Data Loading
5. Model Architecture
6. Optimizers
7. Tokenizer-Agnostic Evaluation (BPB)
8. Post-Training Quantization
9. The Training Loop
10. Test-Time Training with LoRA
11. Serialization and Final Evaluation
12. MLX Differences

Training Logic Walkthrough

A step-by-step explanation of `train_gpt.py` (PyTorch/GPU) and how `train_gpt_mlx.py` (MLX/Mac) differs. Written for someone with machine learning background who is new to deep learning and transformers.

Table of Contents

1. Big Picture
2. Background: What Is a Language Model?
3. Hyperparameters
4. Data Loading
5. Model Architecture
6. Optimizers
7. Tokenizer-Agnostic Evaluation (BPB)
8. Post-Training Quantization
9. The Training Loop
10. Test-Time Training with LoRA
11. Serialization and Final Evaluation
12. MLX Differences

1. Big Picture

The goal: train a language model that **fits in 16MB** and achieves the lowest **bits-per-byte (BPB)** on a held-out FineWeb validation set, all within **10 minutes on 8×H100 GPUs**.

The training pipeline is:

```
Binary token shards → Data loader → Transformer model → Cross-entropy loss →  
Backprop → Optimizer step  
    ↓ (after training)  
Quantize to int8 + zlib compress → Must fit in 16MB  
    ↓  
Evaluate: standard forward pass → BPB score  
Evaluate: LoRA test-time training → BPB score (the competition metric)
```

In plain terms: we read text data that's already been converted to numbers (tokens), feed it through a neural network that tries to predict the next token, measure how wrong it is, and adjust the network's weights to be less wrong. After training, we compress the weights and measure how well the compressed model predicts unseen text.

2. Background: What Is a Language Model?

If you're coming from classical ML (regression, SVMs, random forests), here's how language models map to familiar concepts:

The task: Given a sequence of words (really, tokens), predict the next word. This is fundamentally a **classification problem** — the “classes” are all possible tokens in the vocabulary (1024 in this case).

Tokens: Text is broken into small pieces called tokens. A tokenizer might split “unhappiness” into [“un”, “happiness”]. Each token gets a numeric ID. The vocabulary here is only 1024 tokens (tiny by modern standards — GPT-4 uses ~100k), which keeps the model small.

The model (transformer): A neural network that takes in a sequence of token IDs and outputs, for each position, a probability distribution over what the next token should be. The key innovation is **self-attention** — a mechanism that lets each token look at all previous tokens to decide what comes next.

Cross-entropy loss: The standard loss function for classification. If the model assigns probability p to the correct next token, the loss is $-\log(p)$. Lower loss = the model assigns higher probability to the right answer. The units are “nats” (natural log) — we later convert to bits for the BPB metric.

Backpropagation: The algorithm for computing how each weight contributed to the loss, so we know which direction to adjust each weight. Same concept as computing gradients in logistic regression, just applied through many layers.

Residual connections: A pattern where each layer's output is *added* to its input: `output = input + layer(input)`. This means a layer only needs to learn the *difference* (residual) from its input, which makes training much easier for deep networks. Without residuals, gradients would shrink to near-zero by the time they reach early layers (the “vanishing gradient” problem).

Precision types: Numbers in computers have different precision levels: - **float32** (FP32): 32 bits per number, very precise — used for optimizer state - **bfloat16** (BF16): 16 bits, same range as FP32 but less precise — used during computation for speed (GPUs have specialized hardware for 16-bit math) - **int8**: 8 bits, integers only, range [-128, 127] — used after training for compression

3. Hyperparameters

Lines 39–95 define every tunable knob as a class with defaults, all overridable via environment variables (no CLI args).

Key groups:

Group	Examples	What they control
Model shape	<code>NUM_LAYERS=9 ,</code> <code>MODEL_DIM=512 ,</code> <code>NUM_HEADS=8 ,</code> <code>NUM_KV_HEADS=4</code>	How many layers, how wide each layer is, how attention is structured
Training length	<code>ITERATIONS=20000 ,</code> <code>MAX_WALLCLOCK_SECONDS=600</code>	How long to train (whichever limit hits first)
Batch size	<code>TRAIN_BATCH_TOKENS=524288 ,</code> <code>TRAIN_SEQ_LEN=1024</code>	How many tokens the model sees before each weight update
Learning rates	<code>EMBED_LR ,</code> <code>MATRIX_LR ,</code> <code>SCALAR_LR</code>	How big each weight update is (different LRs for different parameter types — explained in Optimizers)
LoRA TTT	<code>TTT_LORA_RANK=8 ,</code> <code>TTT_LORA_LR=0.01</code>	Settings for the test-time adaptation trick at evaluation (explained in section 10)

Batch size intuition: Training on one example at a time gives noisy gradient estimates. Training on many examples at once (a “batch”) averages out the noise, giving a more reliable direction to update weights. 524,288 tokens is a large batch — about 500 sequences of 1024 tokens each.

4. Data Loading

What the data looks like

The training text (from the FineWeb dataset) has already been preprocessed into **tokens** — integer IDs from 0 to 1023. For example, the sentence “The cat sat” might become `[42, 517, 203]`.

Shard format (lines 436–450)

The tokens are stored across multiple binary files (“shards”) named `fineweb_train_000.bin`, `fineweb_train_001.bin`, etc. Each file has: - A **256-integer header** (contains a magic number to verify the file is valid, a version number, and the token count) - Then the raw **uint16 token IDs** packed back-to-back with no separator

`load_data_shard()` reads one file and returns a flat array of token IDs.

TokenStream (lines 453–481)

A simple sequential reader. It doesn’t shuffle or sample randomly — it reads tokens in order: 1. Start with the first shard file 2. Read tokens sequentially via `.take(n)` — “give me the next n tokens” 3. When a shard runs out, move to the next file 4. After the last file, wrap around to the first (starting a new “epoch”)

Why no shuffling? In classical ML, shuffling training data is important for SGD convergence. Language models handle this differently — the FineWeb dataset is already a diverse collection of web documents in no particular order, so sequential reading provides enough variety. Shuffling at the token level would destroy the sequential structure that language models need to learn.

DistributedTokenLoader (lines 484–501)

When training on multiple GPUs, each GPU needs its own slice of data. This loader: 1. Reads a big chunk of tokens from the stream (enough for all GPUs) 2. Gives each GPU a different, non-overlapping slice 3. Each GPU reshapes its slice into input/target pairs

The input/target pairs are created by **shifting by one token**:

Raw tokens: [The, cat, sat, on, the, mat, ...]

Input (x): [The, cat, sat, on, the] ← "given these tokens..."

Target (y): [cat, sat, on, the, mat] ← "...predict these next tokens"

This is how language models learn: for every position, the model sees all tokens up to that point and tries to predict what comes next. The `+1` in the code (`per_rank_span = local_tokens + 1`) is because you need $N+1$ raw tokens to create N input-target pairs.

5. Model Architecture

Prerequisites: What is a Transformer?

A transformer is a neural network made of stacked **layers** (called "blocks"). Each block has two main parts:

1. **Self-attention**: Lets each token look at all previous tokens and decide which ones are relevant for predicting the next token. For example, in "The cat sat on the ____", the word "cat" is highly relevant for predicting "mat", even though they're far apart.
2. **MLP (feedforward network)**: A simple two-layer network that processes each token independently. If attention is about *communication between tokens*, the MLP is about *processing within each token*.

Each block also uses **residual connections** (explained in Background) and **normalization** to keep values in a stable range.

Overview of this specific model

Input token IDs

↓

Token Embedding (vocab_size × model_dim)

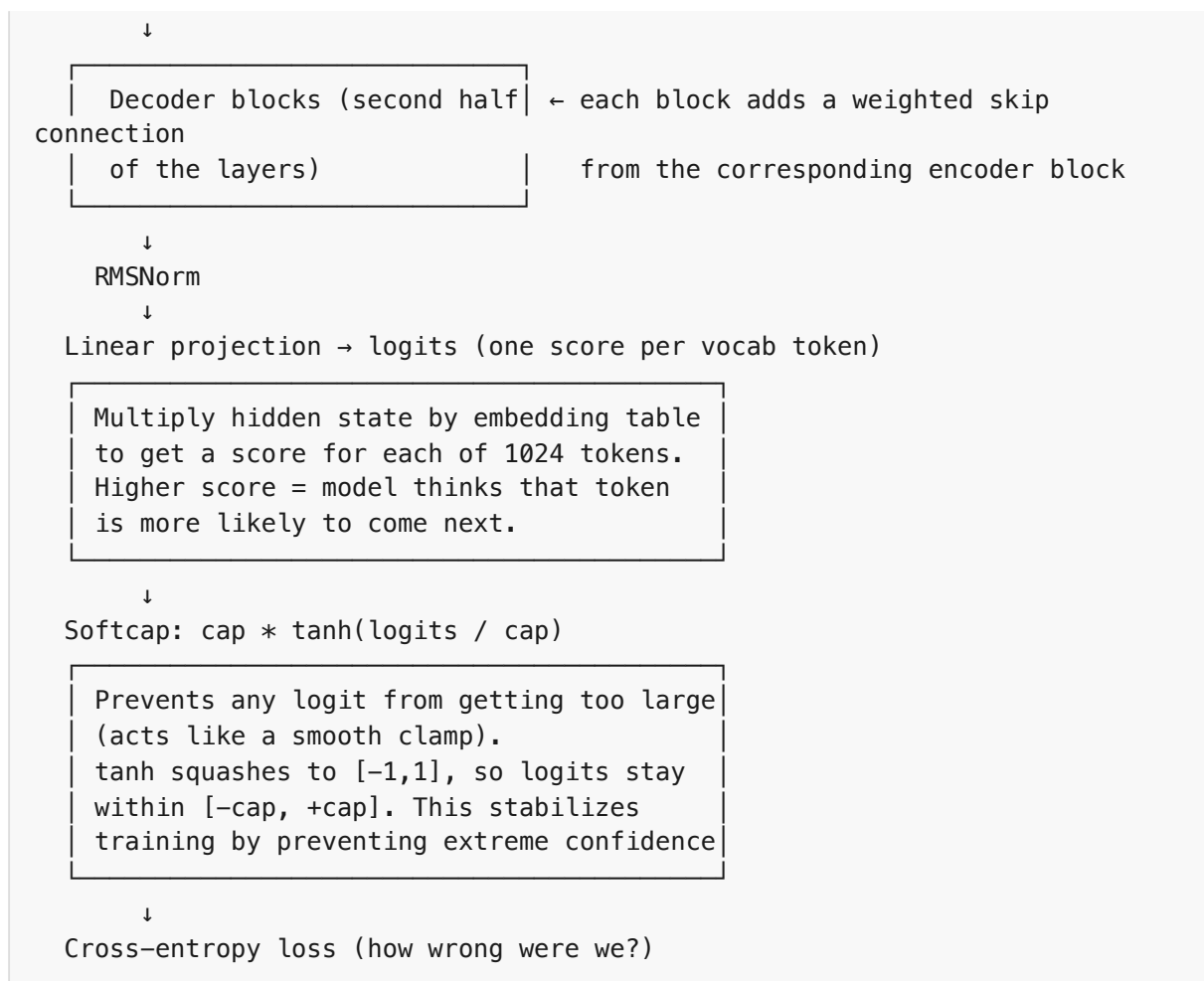
Each token ID (e.g., 42) gets looked up in a table to get a vector of 512 numbers – the token's "meaning" in a form the network can process.

↓

RMSNorm (normalize the vectors)

↓

Encoder blocks (first half) ← each block saves its output to a "skip" stack



Individual components

RMSNorm (line 507)

A normalization layer. In classical ML terms, think of it like feature scaling — it keeps the numbers in each layer within a reasonable range so that training is stable.

Unlike LayerNorm (which subtracts the mean and divides by std), RMSNorm only divides by the root-mean-square:

$$\text{RMSNorm}(x) = x / \sqrt{\text{mean}(x^2) + \text{eps}}$$

No learnable scale/bias parameters here — keeps the model small. The `eps` prevents division by zero.

CastedLinear (line 516)

A linear layer ($y = xW$) with a precision trick: - **Stores weights in float32** — the optimizer needs high precision to make small, accurate updates - **Casts to bfloat16 during computation** — the GPU has specialized hardware (tensor cores) that does bfloat16 math ~2× faster than float32

This is called “mixed precision” training: high precision where it matters (weight storage/updates), low precision where speed matters (matrix multiplications).

Rotary Positional Embeddings — RoPE (lines 531–559)

The problem: A transformer’s self-attention treats input as a *set* — it has no inherent sense of token order. “The cat sat” and “sat cat The” would produce the same attention pattern without positional information. But word order clearly matters in language!

The solution: RoPE encodes position by **rotating** the query and key vectors. Each position gets a different rotation angle, and the angle varies across dimensions:

```
For position t and dimension pair (2i, 2i+1):
    angle = t / (base^(2i/dim))    ← higher dimensions rotate more slowly
```

The rotation:

$$\begin{bmatrix} x_{2i} \\ x_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos(\text{angle}) & \sin(\text{angle}) \\ -\sin(\text{angle}) & \cos(\text{angle}) \end{bmatrix} \begin{bmatrix} x_{2i} \\ x_{2i+1} \end{bmatrix}$$

Why rotations? When the model computes $Q \cdot K^T$ (the attention score), the dot product between a rotated query at position t and a rotated key at position s depends only on $t - s$ (the relative distance). So the model learns “this token is 3 positions before that one” rather than “this token is at position 7” — relative positions generalize much better.

Intuition: Imagine each token’s vector as an arrow in space. RoPE rotates each arrow by an amount proportional to its position. Two arrows at nearby positions end up pointing in similar directions (high dot product = high attention), while distant positions point differently (low attention). Different dimensions rotate at different speeds, like the hands of a clock — some capture short-range relationships, others capture long-range ones.

The Rotary module pre-computes and caches the cos/sin tables so they don’t need to be recomputed each forward pass.

CausalSelfAttention (lines 562–613)

This is the attention mechanism — the core of the transformer. Here’s what happens step by step:

1. **Project** input x into three different representations through linear layers:
 - **Queries (Q):** “What am I looking for?”
 - **Keys (K):** “What do I contain?”
 - **Values (V):** “What information do I provide?”

2. **Reshape** into multiple “heads”. With `num_heads=8`, the 512-dim vector is split into 8 independent 64-dim attention computations. Each head can learn to focus on different aspects (one head might focus on syntax, another on topic, etc.)
3. **Normalize** Q and K with RMSNorm — this stabilizes the dot products and allows using larger learning rates without training blowing up
4. **Apply RoPE** to Q and K — encodes position information (see above)
5. **Scale Q** by a learnable per-head gain (`q_gain`, initialized to 1.5) — controls how “sharp” the attention distribution is. Higher gain → more focused attention; lower → more diffuse
6. **Compute attention scores**: For each token, compute `softmax(Q · K^T / sqrt(head_dim))`:
 - `Q · K^T` measures similarity between every pair of tokens
 - `/ sqrt(head_dim)` prevents the dot products from getting too large (which would make softmax outputs nearly one-hot)
 - `softmax` turns scores into a probability distribution — how much to attend to each previous token
 - A **causal mask** ensures each token can only attend to tokens that came *before* it (you can’t peek at the future when predicting)
7. **Weighted sum**: Multiply attention probabilities by V to get a weighted combination of values from relevant tokens
8. **Project** the output back to model dimension through a final linear layer

Grouped Query Attention (GQA): With `num_heads=8` and `num_kv_heads=4`, every 2 query heads share the same Key and Value head. This saves ~30% of the parameters in attention layers with minimal quality loss. The intuition: queries need to be very specific (“what exactly am I looking for?”) but keys/values can be shared (“here’s roughly what I contain”).

MLP (lines 616–627)

A two-layer feedforward network that processes each token independently:

$$\text{MLP}(x) = W_{\text{proj}} \cdot (\text{ReLU}(W_{\text{fc}} \cdot x))^2$$

Step by step: 1. **Expand**: Project from 512 dims to 1024 dims (the “hidden” layer), via `W_fc`
 2. **Activate with ReLU**: Zero out all negative values. This introduces non-linearity — without it, stacking linear layers would just be one big linear layer
 3. **Square**: Square each value. This is the “ReLU²” activation. Compared to plain ReLU, squaring makes the output sparser and

sharper — many values near zero get pushed even closer to zero, while large values get amplified. This works well for small models 4. **Compress:** Project back from 1024 dims to 512 dims, via `W_proj`

The MLP is where the model stores “factual knowledge” — patterns like “Paris is the capital of ____”. Attention figures out *which* tokens are relevant; the MLP processes *what to do* with that information.

Block (lines 630–658)

One transformer layer, combining attention and MLP with residual connections:

```
# 1. Residual mixing: blend current hidden state with the original embedding
x = mix[0] * x + mix[1] * x0

# 2. Attention with residual connection
x = x + attn_scale * Attention(RMSNorm(x))

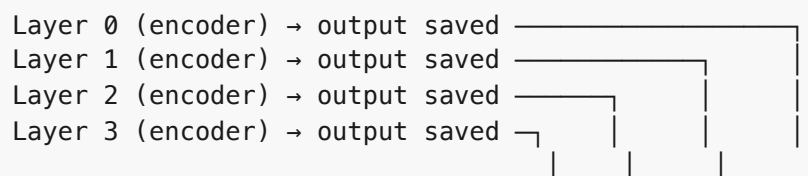
# 3. MLP with residual connection
x = x + mlp_scale * MLP(RMSNorm(x))
```

Three things to note:

- **resid_mix** : Each layer can blend its input with the *original* embedding (`x0` from the very first layer). This is like a shortcut that lets deep layers directly access the raw token information, bypassing all intermediate transformations. `mix[0]` and `mix[1]` are learnable — the model decides how much to rely on recent computation vs. original input. Initialized to `[1, 0]` (100% recent, 0% original), so it starts as a regular residual stream.
- **attn_scale** and **mlp_scale** : Learnable per-dimension scalars (512 values each) that control how much each sublayer contributes. Think of them as volume knobs for each feature dimension. Initialized to 1.0 (equal contribution).
- **Pre-norm pattern**: `RMSNorm(x)` is applied *before* each sublayer, not after. This is a design choice that makes training more stable for deep networks.

GPT (lines 661–743)

The full model. The key architectural choice is the **encoder-decoder skip connections** (inspired by U-Net from image processing):



```

Layer 4 (decoder) ← skip from 3
Layer 5 (decoder) ← skip from 2
Layer 6 (decoder) ← skip from 1
Layer 7 (decoder) ← skip from 0
Layer 8 (decoder, no skip - odd number of layers)

```

Each skip connection is weighted by a learnable scalar (`skip_weights`). The decoder can learn how much to rely on each encoder layer's output.

Why skip connections? In deep networks, information from early layers can get “washed out” by the time it reaches the final layer. Skip connections provide a direct highway for information to flow from early to late layers. The U-Net pattern specifically helps because early layers capture low-level features (character patterns, common phrases) while later layers capture high-level features (grammar, meaning). The decoder benefits from having both.

Tied embeddings: The same weight matrix is used for both the input embedding (token ID → vector) and the output projection (vector → token scores). This halves the parameter cost of embeddings and works because the embedding space is meaningful in both directions — similar tokens should have similar vectors, whether you're encoding or decoding.

Loss computation: Standard cross-entropy — for each position, compute $-\log(\text{probability of correct next token})$, averaged over all positions in the batch.

6. Optimizers

In classical ML, you might use one optimizer (SGD, Adam) for all parameters. This model splits parameters into **three groups**, each with its own optimizer and learning rate. Why? Different parameter shapes and roles benefit from different update strategies.

Group 1: Token Embeddings → Adam

The embedding matrix (1024 tokens × 512 dimensions) maps token IDs to vectors. Trained with standard **Adam** at `TIED_EMBED_LR` (default 0.05 when tied).

Quick Adam refresher: Adam maintains two running averages for each parameter: - **First moment** (mean of recent gradients) — captures the direction of the gradient - **Second moment** (mean of recent squared gradients) — captures the magnitude

The update is: `param -= lr * first_moment / sqrt(second_moment)`. This auto-scales the learning rate per parameter — parameters with consistently large gradients get smaller updates, and vice versa. It's like SGD but with automatic per-parameter learning rate tuning.

Group 2: Matrix Parameters (2D weights in transformer blocks) → Muon

All weight matrices inside attention (Q, K, V, output projections) and MLP layers use **Muon** (lines 97–175), a specialized optimizer designed for matrix-shaped parameters.

The intuition behind Muon: In a standard optimizer, the gradient update can stretch or squish the weight matrix in arbitrary ways — making some singular values very large and others very small. This can lead to training instability. Muon constrains each update to be an **orthogonal matrix** (a “rotation” in high-dimensional space), which preserves the matrix’s singular values. Think of it as: instead of deforming the weight matrix, Muon only *rotates* it.

How Muon works, step by step:

1. **Get the gradient** `g` for each weight matrix
2. **Apply momentum:** Like a ball rolling downhill, momentum accumulates past gradients to build up speed in consistent directions:

```
buf = 0.95 * buf + g          ← running average of past gradients
g_eff = g + 0.95 * buf       ← "Nesterov" look-ahead: peek at where
                             momentum is taking us
```

Nesterov momentum is slightly better than standard momentum because it corrects course before overshooting.

3. **Orthogonalize via Newton-Schulz iteration** (lines 103–116): This is the key step. Given the gradient matrix `g_eff`, find the nearest orthogonal matrix. The algorithm runs 5 iterations of:

```
A = X · X^T
X ← 3.4445 · X + (-4.7750 · A + 2.0315 · A²) · X
```

This converges to the “matrix sign” of `g_eff` — the nearest matrix with all singular values equal to 1. The coefficients (3.4445, -4.7750, 2.0315) are optimized for fast convergence.

Analogy: If the gradient says “stretch this direction 3× and that direction 0.5×”, Muon normalizes both to 1× — it keeps the *directions* but makes the *magnitudes* uniform.

4. **Scale correction:** For non-square matrices, multiply by `sqrt(max(rows, cols) / min(rows, cols))` to account for the aspect ratio
5. **Update:** `W ← W - lr * orthogonalized_gradient`

Distributed work-splitting: In multi-GPU training, the Muon updates are split round-robin across GPUs (GPU 0 handles parameters 0, 8, 16...; GPU 1 handles 1, 9, 17...; etc.), then combined via all-reduce. This parallelizes the expensive Newton-Schulz iterations.

Group 3: Scalar/Control Parameters → Adam

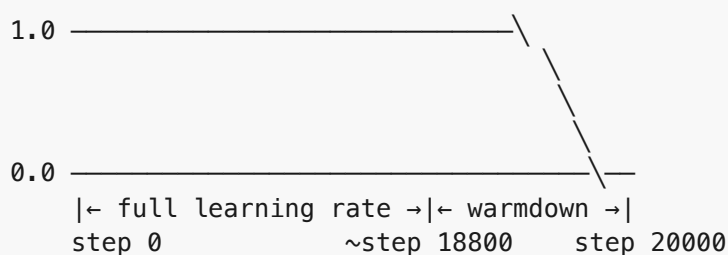
Small parameters that control the model's behavior: - `attn_scale`, `mlp_scale` — volume knobs for each sublayer (512 values each) - `resid_mix` — blending weights between current and original representations - `q_gain` — attention sharpness per head (8 values) - `skip_weights` — strength of U-Net skip connections

These are trained with Adam at `SCALAR_LR`. They don't benefit from Muon (which is designed for large 2D matrices).

Learning rate schedule

All three groups share a **warmdown schedule** (lines 1156–1165). The idea:

LR multiplier over time:



- **Normal phase:** LR multiplier = 1.0 (full learning rate)
- **Warmdown phase:** LR linearly decays to 0

Why decay the LR? Early in training, the model needs big updates to learn basic patterns. Late in training, it's fine-tuning — big updates would overshoot the optimum. Decaying to zero is like slowing down as you approach your destination.

Wallclock-based triggering: The clever part — instead of starting warmdown at a fixed iteration, the script estimates: “At my current speed, how many more steps can I fit in 10 minutes?” If the answer is $\leq \text{warmdown_iters}$, it starts decaying. This way the LR always reaches ~0 right at the time limit, regardless of how fast each step is.

7. Tokenizer-Agnostic Evaluation (BPB)

The problem with raw loss

Cross-entropy loss depends on vocabulary size. If your tokenizer has 1024 tokens, the word “information” might be one token. If it has 256 tokens, that word might be 4 tokens. The 1024-token model predicts 1 harder classification; the 256-token model predicts 4 easier ones. Raw loss values aren’t comparable.

Bits-per-byte normalizes this

Bits-per-byte (BPB) measures: *on average, how many bits does the model need to encode one byte of raw text?* This is independent of tokenizer choice.

$$\text{BPB} = \frac{(\text{avg_loss_per_token} / \ln(2)) \times (\text{total_tokens} / \text{total_bytes})}{\text{converts nats} \rightarrow \text{bits per token} \quad \text{adjusts for tokenizer efficiency}}$$

Example: If your tokenizer is very efficient (each token represents ~3 bytes on average), you have fewer tokens to predict but each prediction covers more bytes. The `tokens/bytes` ratio corrects for this.

A good language model achieves ~1.0–1.5 BPB on web text. For reference, uncompressed English text is ~8 bits per byte (1 byte = 1 character); gzip gets ~2–3 BPB; this challenge’s best models achieve ~1.17 BPB.

Building the lookup tables (lines 187–211)

To compute BPB, we need to know how many bytes each token represents. For each token in the vocabulary:

- `base_bytes_lut[token_id]` = how many UTF-8 bytes this token’s text uses
- `has_leading_space_lut[token_id]` = whether the token starts with SentencePiece’s space marker `▯`
- `is_boundary_token_lut[token_id]` = whether this is a special token (control, unknown, unused)

The leading-space subtlety: SentencePiece encodes word boundaries as `▯` prefixed to the following token. “The cat” becomes `["▯The", "▯cat"]`. The `▯` represents a space character (1 byte), but we only count it when it actually corresponds to a space in the original text — specifically, when the previous token is a normal (non-boundary) token. This logic is in line 273:

```
token_bytes += (has_leading_space_lut[target] &
               ~is_boundary_token_lut[previous])
```

Evaluation loop (lines 226–285)

1. Split the validation data across GPUs (each GPU evaluates a different slice)
2. For each batch: run the model forward **without computing gradients** (saves memory and time)
3. Accumulate: total cross-entropy loss, total tokens processed, total bytes those tokens represent
4. All-reduce across GPUs (sum up each GPU's partial counts)
5. Final BPB = $(\text{total_loss} / \ln(2)) / \text{total_bytes}$

8. Post-Training Quantization

Why quantize?

After training, the model weights are in bfloat16 (2 bytes per value) and float32 (4 bytes per value). A model with ~4.5M parameters in bf16 would be ~9MB — already tight for the 16MB limit when you add code. Quantization reduces each weight to **1 byte** (int8), roughly halving the size, with a small accuracy cost.

How int8 quantization works (lines 288–429)

The core idea: map floating-point weights (which can be any real number) to integers from -127 to +127, using a scale factor.

```
Quantize:  q = round(value / scale), clamped to [-127, 127]
Dequantize: value ≈ q × scale
```

The **scale** determines the mapping range. For example, if `scale = 0.01`, then the range [-1.27, +1.27] maps to integers [-127, +127]. Values outside this range get clipped to ±127.

Different strategies for different tensor shapes:

2D matrices (the large weight tensors) — per-row quantization: 1. For each row, compute the 99.99984th percentile of absolute values → `clip_abs` (this is ~4.5 standard deviations — clips only extreme outliers) 2. Clip the row's values to `[-clip_abs, clip_abs]` 3. Scale = `clip_abs / 127` (maps the clipped range to [-127, 127]) 4. Quantize: `q = round(value / scale)` 5. Store: `q` as int8 (1 byte per value) + one float16 scale per row (2 bytes per row)

Per-row is better than per-tensor because different rows (output neurons) can have very different value ranges. A single global scale would waste precision on rows with small values.

Vectors and scalars — per-tensor quantization: - Same math but one scale for the entire tensor (simpler, and these tensors are small)

Small tensors ($\leq 65,536$ elements) — kept as float16: - The overhead of storing int8 + scales isn't worth it for small tensors

Control tensors (`attn_scale`, `mlp_scale`, `skip_weights`, etc.) — kept as float32: - These are tiny (~512 values each) but very sensitive to precision loss. A small error in `attn_scale` affects every token in every sequence.

The full compression pipeline

```
Model weights (bf16/fp32, ~9MB)
  ↓ int8 quantization
Quantized dict (int8 values + fp16 scales, ~5MB uncompressed)
  ↓ torch.save() serializes to bytes
Serialized bytes
  ↓ zlib level 9 (standard compression, like gzip)
Final artifact (.int8.ptz, ~3-4MB)
```

zlib works well here because int8 weights have limited entropy (only 256 possible values per weight) and nearby weights often have similar values.

Dequantization (lines 408–429)

To use the quantized model at evaluation time: 1. Read the `.int8.ptz` file 2. Decompress with zlib 3. For each quantized tensor: `reconstructed = int8_values.float() × scale` 4. Cast back to the original dtype 5. Load into the model

The reconstructed weights aren't exactly the same as the originals — there's quantization error. Typically this costs ~0.001–0.005 BPB.

9. The Training Loop

Phase 1: Setup (lines 961–1050)

1. **Distributed setup:** If running on multiple GPUs, initialize NCCL (NVIDIA's GPU-to-GPU communication library). Compute `grad_accum_steps = 8 / world_size` — explained below.
2. **CUDA optimizations:** Enable TF32 (a slightly-reduced-precision float32 that's faster on modern GPUs) and Flash Attention (a memory-efficient attention algorithm).

3. **Seed everything:** Set random seeds for numpy, torch, and CUDA. This makes training reproducible — same seed → same results.
4. **Load tokenizer** and build the BPB lookup tables.
5. **Load validation tokens** entirely into GPU memory (they're small enough).
6. **Create model:** Build the GPT, convert to bfloat16, keep CastedLinear weights in float32, then call `torch.compile()`:
 - `torch.compile` traces the model's computation graph and generates optimized CUDA kernels (fusing multiple small operations into fewer, faster ones). This gives ~20-50% speedup but takes time to compile the first time.
7. **Wrap in DDP** (DistributedDataParallel) if multi-GPU: DDP automatically synchronizes gradients across GPUs during backpropagation.
8. **Create the three optimizers** (Adam for embeddings, Muon for matrices, Adam for scalars).

Phase 2: Warmup (lines 1169–1193)

`torch.compile` is lazy — it doesn't actually compile until the first forward pass. The first few steps are therefore very slow. The warmup phase handles this:

1. **Save** the initial model weights and all optimizer states (a full snapshot)
2. Run `WARMUP_STEPS` (default 20) complete training steps — forward pass, backward pass, optimizer update. This triggers compilation.
3. **Restore** the saved snapshot — weights, optimizer states, everything goes back to the initial state
4. **Reset** the data loader to the beginning of the data

The training timer starts *after* warmup, so compilation time doesn't count against the 10-minute limit. The model begins training from its true initial weights, with pre-compiled fast kernels.

Phase 3: Main loop (lines 1199–1288)

Here's the simplified flow:

```
while step < iterations and not hit_wallclock_cap:
    # --- Validation check ---
    if time_to_validate:
        pause_training_timer()
        val_loss, val_bpb = evaluate_on_validation_set()
        log_results()
```



```
resume_training_timer()

# --- Compute learning rate ---
scale = lr_schedule(step, elapsed_time) # 1.0 normally, decays to 0 in
warmdown

# --- Forward + backward (with gradient accumulation) ---
zero_all_gradients()
for micro_step in range(grad_accum_steps):
    x, y = get_next_batch()
    loss = model.forward(x, y)          # predict next tokens,
    measure error                       # compute gradients, scaled
    (loss / grad_accum_steps).backward() down

# --- Update Muon momentum (gradually increases early in training) ---
muon_momentum = interpolate(0.85 → 0.95, over first 500 steps)

# --- Apply LR schedule to all optimizers ---
for optimizer in all_optimizers:
    optimizer.lr = base_lr * scale

# --- Update weights ---
for optimizer in all_optimizers:
    optimizer.step() # Adam or Muon update

# --- Check if we've hit the 10-minute wall ---
if elapsed_time >= 600 seconds:
    stop_after_step = current_step # finish this step, validate, then
    exit
```

Gradient accumulation explained

The competition requires a large batch size (524,288 tokens) for good training dynamics, but a single GPU can't fit that many tokens in memory at once.

Solution: Process the batch in smaller chunks ("microbatches"), accumulating gradients:

Scenario	World size	Grad accum steps	Tokens per GPU per micro-step
8×H100	8	1	65,536
4×H100	4	2	65,536
1×H100	1	8	65,536

Each micro-step: 1. Forward pass on 65,536 tokens → compute loss 2. Backward pass → compute gradients and *add* them to the existing gradient buffers 3. Scale the loss by `1/grad_accum_steps` so the total gradient magnitude is the same regardless of how many micro-steps

After all micro-steps, the accumulated gradient is mathematically identical to what you'd get from processing the full 524,288 tokens at once.

DDP gradient sync timing

With multiple GPUs, gradients need to be averaged across GPUs (all-reduce). But doing this after every micro-step would be wasteful — we only need the final accumulated gradient.

```
model.require_backward_grad_sync = (micro_step == grad_accum_steps - 1)
```

This tells DDP: “Only synchronize on the last micro-step.” Earlier micro-steps accumulate gradients locally on each GPU, saving inter-GPU communication.

The wallclock cap

The `stop_after_step` mechanism ensures graceful shutdown: 1. After each step, check: have we exceeded 600 seconds? 2. If yes on *any* GPU, set `stop_after_step = current_step` 3. The `all_reduce(MAX)` ensures all GPUs agree (even if one GPU is slightly faster) 4. The loop runs one more iteration (to do final validation), then exits

10. Test-Time Training with LoRA

After standard evaluation, the model gets a **second evaluation** using **Test-Time Training (TTT)** with **LoRA adapters** (lines 746–955). This second score is the actual competition metric.

What is LoRA?

Low-Rank Adaptation adds a small, trainable “patch” to each weight matrix without modifying the original weights:

```
Without LoRA:  output = input × W
With LoRA:     output = input × W + input × A × B
                                   └─ the LoRA patch ─┘
```

- `W` is the original weight matrix ($512 \times 512 = 262,144$ parameters) — **frozen** (not updated)
- `A` is a small “down-projection” matrix ($512 \times 8 = 4,096$ parameters)
- `B` is a small “up-projection” matrix ($8 \times 512 = 4,096$ parameters)

The product $A \times B$ gives a 512×512 matrix, but it's **low-rank** (rank 8) — it can only express a limited subspace of changes. This is enough to adapt the model's behavior with only ~3% of the original parameter count.

B is initialized to zero, so the LoRA patch starts as zero (the model behaves exactly as before). During training, A and B learn a useful correction.

What is Test-Time Training?

Standard evaluation: load model → run on test data → get score. The model is frozen.

Test-time training: load model → for each test document, *briefly train* the model on the beginning of that document → then score it on the rest. This lets the model adapt to each document's specific style, vocabulary, and patterns.

Analogy: Imagine taking an exam. Standard evaluation = you study once, then answer all questions. Test-time training = for each question, you get to read a relevant textbook chapter first, then answer. You'll do better because you've refreshed your memory on the specific topic.

The key insight: **the LoRA weights are reset between documents**. Each document gets a fresh adaptation. The base model (trained on all the data) provides general knowledge; LoRA provides per-document specialization.

Why score *before* training on each chunk?

For each chunk of a document: 1. First, **score** the model's predictions (record the loss) — this is the “test” part 2. Then, **train** the LoRA on this chunk — this helps with future chunks

If we trained first and scored second, we'd be evaluating on data the model has already seen — that's cheating (data leakage). By scoring first, we ensure the model is always evaluated on tokens it hasn't been trained on yet.

The full TTT-LoRA flow (lines 848–955)

```
For each batch of 64 documents:
    Reset all LoRA weights to initial values

    For each chunk (256 tokens at a time):
        1. Build context window (up to 1024 tokens ending at this chunk)
        2. Forward pass with LoRA → get per-token losses
        3. SCORE: Record this chunk's losses for BPB calculation
        4. TRAIN (if not the last chunk): Backprop through LoRA, one Adam
step

    → LoRA has now adapted to this document's patterns
```

→ But we only counted losses from before each training step (no data leakage)

Context windowing

Each chunk is evaluated within a sliding context window. The model sees up to

`TTT_EVAL_SEQ_LEN` (1024) tokens of context:

```
Document: [tok0, tok1, ..., tok2047]
Chunk size = 256, Window size = 1024

Chunk 0: window = [0:256],      score [0:256]      ← short context (only 256
tokens)
Chunk 1: window = [0:512],      score [256:512]    ← growing context
Chunk 2: window = [0:768],      score [512:768]
Chunk 3: window = [0:1024],     score [768:1024]   ← full context window
Chunk 4: window = [256:1280],   score [1024:1280]  ← window starts sliding
Chunk 5: window = [512:1536],   score [1280:1536]  ← keeps sliding
...
```

Early chunks have less context (the document just started). Once the document is longer than the window size, the window slides forward, always ending at the current chunk. This means later chunks benefit from more context *and* from LoRA having been trained on earlier chunks.

Batched processing (lines 884–946)

Processing one document at a time would be very slow. Instead:

1. **Sort documents by length** — batches of similar-length docs waste less computation on padding
2. **Process 64 documents simultaneously** — the LoRA tensors have a batch dimension:
`A` is shape `(64, rank, dim)` — 64 independent LoRA adapters in parallel
3. For the last batch (which might have fewer than 64 docs), create a smaller LoRA module to avoid wasting memory
4. **Mask for partial batches:** When documents have different lengths, some finish before others. A mask tensor zeros out the loss for finished documents so they don't interfere with the Adam update for still-active documents

11. Serialization and Final Evaluation

After training completes (lines 1300–1368), the script runs three evaluations:

1. **Save raw model** as `final_model.pt` — unquantized, useful for debugging or further training
 2. **Quantize + compress** → `final_model.int8.ptz` :
 - int8 quantization (reduce precision)
 - `torch.save` (serialize to bytes)
 - zlib level 9 (lossless compression)
 - Log the file size (must be < 16MB including code)
 3. **Roundtrip validation** — load the quantized model back and run standard eval:
 - This confirms the quantization didn't break the model
 - You see the quality gap: e.g., “pre-quantization BPB: 1.25, post-quantization BPB: 1.26”
 4. **LoRA TTT evaluation** — the final competition score:
 - Load the quantized model
 - Run full test-time training evaluation (all validation documents, with per-document LoRA adaptation)
 - This is typically 0.02–0.05 BPB better than standard eval (the LoRA adaptation helps!)
 5. **Clean up** distributed process group (if multi-GPU)
-

12. MLX Differences

`train_gpt_mlx.py` implements the **same model and training logic** but for Apple Silicon Macs (using Apple's MLX framework). It's meant for **local development and experimentation**, not competition submissions.

Same things

- Model architecture (GPT with skip connections, GQA, RoPE, ReLU² MLP)
- Optimizer split (Adam for embeddings/scalars, Muon for matrices)
- Quantization math (int8 per-row, same clip percentile)
- Hyperparameters (same defaults)

What's different

Aspect	PyTorch (<code>train_gpt.py</code>)	MLX (<code>train_gpt_mlx.py</code>)
Hardware	NVIDIA GPUs (CUDA)	Apple Silicon (Metal)
Multi-GPU	Yes (DDP + NCCL)	No (single device only)
LoRA TTT eval	Yes (the competition score)	No (standard eval only)
Grad accum	8 / <code>world_size</code> (auto)	Explicit env var (default 8)
Compilation	<code>torch.compile()</code> (JIT)	<code>mx.compile()</code> (ahead-of-time)
Serialization	<code>torch.save()</code> + <code>zlib</code>	<code>pickle.dumps()</code> + <code>zlib</code>
Tied embed only	Optional	Required (untied not supported)

MLX-specific: Lazy evaluation and memory control

The biggest conceptual difference is MLX's **lazy evaluation model**:

- **PyTorch**: Operations execute immediately. `a + b` computes the result right away.
- **MLX**: Operations are deferred. `a + b` records “I need to add these” but doesn't compute anything yet. The actual computation happens only when you call `mx.eval()` or access a value (like `.item()`).

Why this matters for memory: Without intervention, MLX would build up a massive computation graph across all gradient accumulation steps, then try to execute it all at once — potentially exceeding available RAM.

Two knobs control this:

1. **MLX_MAX_MICROBATCH_TOKENS** (default 8192): Further splits each microbatch into sub-batches. On a 16GB Mac, you can't fit 65,536 tokens in one computation graph. Breaking it into 8 chunks of 8,192 keeps peak memory manageable.
2. **MLX_EAGER_EVAL** (default on): Calls `mx.eval()` after each sub-batch, forcing MLX to actually compute and free the graph. This trades some throughput for much lower peak memory. On a 32GB+ Mac, you can disable this for better speed.

MLX-specific: Compilation and state capture

MLX's `mx.compile()` requires you to explicitly declare which state (model parameters, optimizer buffers) the compiled function reads and writes:

```
compiled_loss = mx.compile(  
    lambda x, y: model.loss(x, y),  
    inputs=model.state,      # ← "this function reads model parameters"  
    outputs=model.state,    # ← "this function may modify model state"  
)
```

Two separate functions are compiled: - `compiled_loss` — for evaluation (forward pass only)
- `compiled_loss_and_grad` — for training (forward + backward)

These have different computation graphs, so they need separate compilation.

MLX-specific: Warmup behavior

PyTorch warmup saves and restores the full model+optimizer state. MLX warmup is simpler — it runs forward/backward passes to trigger compilation but **doesn't restore weights**. Saving and restoring the full state on unified memory (where CPU and GPU share RAM) is expensive, and since MLX is for local development only, the slight weight modification from warmup steps is acceptable.